

Blockchain aplikacije

Fakultet informatike, Sveučilište Jurja Dobrile u Puli

izv. prof. dr. sc. Nikola Tanković

Table of contents

Blockchain aplikacije 6

1 Od raspodijeljenih do decentraliziranih sustava 7
1.1 Vodič kroz predavanje . . . 7
1.2 Kako surađuju banke s odvojenim evidencijama? . . . 7
1.3 Pravni put postoji — koliko traje i košta? . . . 8
1.4 Možemo li dio sporova spriječiti unaprijed? . . . 9
1.5 Što ćemo moći objasniti . . . 10
1.6 Tri organizacije, jedan digitalni budžet . . . 10
1.7 Stanje zapisujemo izričito . . . 11
1.8 Pravilo koje mora ostati istinito . . . 12
1.9 Prvo rješenje: zajednički koordinator . . . 12
1.10 Dva zahtjeva, jedan redoslijed . . . 13
1.11 Naivan kod ipak može pogriješiti . . . 13
1.12 Jedna baza: atomarnost i izolacija . . . 14
1.13 Raspodijeljenost: rad preko više procesa . . . 15
1.14 Zašto dodajemo replike? . . . 16
1.15 Kopije nisu trenutačno jednake . . . 17
1.16 Čitanje stare kopije mijenja korisnikov doživljaj . . . 17
1.17 Zahtjev je izvršen, potvrda nije stigla . . . 18
1.18 Ponovljeni pokušaj treba isti identitet . . . 19
1.19 Što znači djelomičan ispad? . . . 19
1.20 Prije jamstva: napišimo model . . . 20
1.21 Spor čvor i zaustavljen čvor mogu izgledati isto . . . 21
1.22 Mreža se razdvojila . . . 21
1.23 Dvije strane ne smiju obećati suprotno . . . 22
1.24 Tri različita pitanja o arhitekturi . . . 23
1.25 Sto poslužitelja, jedan administrator . . . 23
1.26 P2P opisuje razmjenu, ne jamči dogovor . . . 24
1.27 Razvrstajte tri sustava . . . 25
1.28 Nova pretpostavka: nema zajedničkog operatora . . . 25
1.29 Povjerenje napišemo kao konkretne ovlasti . . . 26
1.30 Pad i proizvoljno ponašanje nisu isti kvar . . . 27
1.31 Isti pošiljatelj govori dvije stvari . . . 27
1.32 Kašnjenje ili proturječje? . . . 28
1.33 Identitet ne zamjenjuje pravilo prihvaćanja . . . 29
1.34 „Uzmimo dva od tri” još nije protokol . . . 29
1.35 Tko uopće ima pravo glasa? . . . 30

1.36 Radionica: dvije isplate iz istog budžeta . . . 31
1.37 Analiza: gdje se izgubilo jamstvo? . . . 31
1.38 Odaberite arhitekturu prema problemu . . . 32
1.39 Provjerite uvjerljiv AI prijedlog . . . 33
1.40 Sažetak: tri razlike koje ostaju . . . 33
1.41 Sljedeći izazov: dogovor bez zajedničkog šefa . . . 34
1.42 Literatura za produbljivanje . . . 35

2 Konsenzus i problem bizantskih generala 36
2.1 Ishodi učenja . . . 36
2.2 Problem i predznanje . . . 36
2.3 Pojmovi i pretpostavke . . . 36
2.4 Primjer i tijek stanja . . . 36
2.5 Ispad i poboljšani dizajn . . . 36
2.6 Rasprava i provjera razumijevanja . . . 36
2.7 Sažetak i literatura . . . 36

3 Temelji blockchaina 37
3.1 Ishodi učenja . . . 37
3.2 Problem i predznanje . . . 37
3.3 Pojmovi i pretpostavke . . . 37
3.4 Primjer i tijek stanja . . . 37
3.5 Ispad i poboljšani dizajn . . . 37
3.6 Rasprava i provjera razumijevanja . . . 37
3.7 Sažetak i literatura . . . 37

4 Bitcoin i dokaz rada 38
4.1 Ishodi učenja . . . 38
4.2 Problem i predznanje . . . 38
4.3 Pojmovi i pretpostavke . . . 38
4.4 Primjer i tijek stanja . . . 38
4.5 Ispad i poboljšani dizajn . . . 38
4.6 Rasprava i provjera razumijevanja . . . 38
4.7 Sažetak i literatura . . . 38

5 Ethereum i programabilni blockchain 39
5.1 Ishodi učenja . . . 39
5.2 Problem i predznanje . . . 39
5.3 Pojmovi i pretpostavke . . . 39
5.4 Primjer i tijek stanja . . . 39
5.5 Ispad i poboljšani dizajn . . . 39
5.6 Rasprava i provjera razumijevanja . . . 39
5.7 Sažetak i literatura . . . 39

6 Dokaz uloga i moderni Ethereum 40
6.1 Ishodi učenja . . . 40

6.2 Problem i predznanje . . . 40
6.3 Pojmovi i pretpostavke . . . 40
6.4 Primjer i tijek stanja . . . 40
6.5 Ispad i poboljšani dizajn . . . 40
6.6 Rasprava i provjera razumijevanja . . . 40
6.7 Sažetak i literatura . . . 40

7 Pametni ugovori u Solidityju 41
7.1 Ishodi učenja . . . 41
7.2 Problem i predznanje . . . 41
7.3 Pojmovi i pretpostavke . . . 41
7.4 Primjer i tijek stanja . . . 41
7.5 Ispad i poboljšani dizajn . . . 41
7.6 Rasprava i provjera razumijevanja . . . 41
7.7 Sažetak i literatura . . . 41

8 Dizajn i sigurnost pametnih ugovora 42
8.1 Ishodi učenja . . . 42
8.2 Problem i predznanje . . . 42
8.3 Pojmovi i pretpostavke . . . 42
8.4 Primjer i tijek stanja . . . 42
8.5 Ispad i poboljšani dizajn . . . 42
8.6 Rasprava i provjera razumijevanja . . . 42
8.7 Sažetak i literatura . . . 42

9 Razvoj Web3 aplikacija 43
9.1 Ishodi učenja . . . 43
9.2 Problem i predznanje . . . 43
9.3 Pojmovi i pretpostavke . . . 43
9.4 Primjer i tijek stanja . . . 43
9.5 Ispad i poboljšani dizajn . . . 43
9.6 Rasprava i provjera razumijevanja . . . 43
9.7 Sažetak i literatura . . . 43

10 Decentralizirana pohrana i IPFS 44
10.1 Ishodi učenja . . . 44
10.2 Problem i predznanje . . . 44
10.3 Pojmovi i pretpostavke . . . 44
10.4 Primjer i tijek stanja . . . 44
10.5 Ispad i poboljšani dizajn . . . 44
10.6 Rasprava i provjera razumijevanja . . . 44
10.7 Sažetak i literatura . . . 44

11 Arhitekture blockchain aplikacija 45
11.1 Ishodi učenja . . . 45
11.2 Problem i predznanje . . . 45

11.3 Pojmovi i pretpostavke . . . 45
11.4 Primjer i tijek stanja . . . 45
11.5 Ispad i poboljšani dizajn . . . 45
11.6 Rasprava i provjera razumijevanja . . . 45
11.7 Sažetak i literatura . . . 45

12 Skaliranje, ograničenja i kada koristiti blockchain **46**
12.1 Ishodi učenja . . . 46
12.2 Problem i predznanje . . . 46
12.3 Pojmovi i pretpostavke . . . 46
12.4 Primjer i tijek stanja . . . 46
12.5 Ispad i poboljšani dizajn . . . 46
12.6 Rasprava i provjera razumijevanja . . . 46
12.7 Sažetak i literatura . . . 46

Blockchain aplikacije

Plan materijala — nije dovršen udžbenik.

Povjerenje i decentralizacija → konsenzus → kriptografija → Bitcoin → Ethereum → Solidity → Web3 → IPFS → cjelovite aplikacije.

Materijali se prvo razvijaju na hrvatskom. Svih 12 poglavlja zasad su planirani nacrti.

1 Od raspodijeljenih do decentraliziranih sustava

Nacrt za nastavnikov pregled. Predavanje traje 120 minuta s raspravom, zadacima i ponavljanjem. Engleski prijevod slijedi nakon pregleda hrvatske verzije.

1.1 Vodič kroz predavanje

Polazimo od pitanja tko kontrolira evidenciju zajedničkog digitalnog budžeta. Konkurentne isplate 80 i 60 iz salda 100 kasnije služe za analizu promjena stanja. Od jednog koordinatora prelazimo na replike, zatim na kvarove i neovisne operatore. Predznanje: zapis podataka, uvjet i poziv funkcije. Crteži i pseudokod prikazuju nastavne modele; ne predstavljaju gotov distribuirani protokol.

1.2 Kako surađuju banke s odvojenim evidencijama?

Banke ne moraju dati jednoj drugoj administratorsku kontrolu nad svim svojim evidencijama. Vode zasebne zapise, razmjenjuju naloge i koriste dogovorene postupke obračuna obveza i njihova poravnanja. U jednom konkretnom sustavu, T2, međubankovne se obveze poravnavaju prijenosom novca između računa kod središnje banke u stvarnom vremenu. ECB: T2. Sud nije korak kroz koji prolazi svako plaćanje: pravna zaštita služi rješavanju sporova, dok poravnanje ima vlastita pravila i infrastrukturu.

Naš zajednički digitalni budžet pojednostavnjuje pitanje zajednički prihvaćenog stanja; nije model cjelokupnog bankovnog poslovanja. Organizacije mogu prihvatiti operatora, ugovor, reviziju i nadležni sud. Ako zapisi postanu nespojivi, treba prikupiti dokaze, uskladiti evidencije ili riješiti spor. Želimo istražiti može li zajedničko provođenje preciznih pravila spriječiti dio takvih neslaganja prije nego što nastanu.

Zato početno pitanje nije kako računati dvije istodobne isplate. Pitanje je tko smije konačno prihvatiti ili promijeniti zajednički zapis i na čiji se autoritet ostali oslanjaju. Najprije ćemo vidjeti što rješava pouzdan koordinator, zatim replike i na kraju neovisni operatori. Isplate 80 i 60 kasnije daju konkretno pravilo za provjeru tih rješenja.

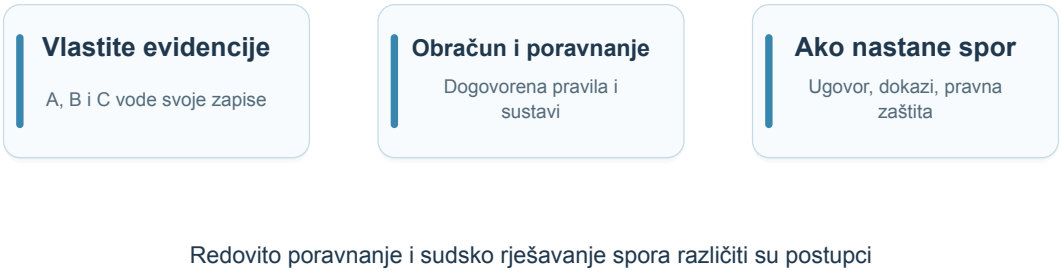


Figure 1.1: Redovito poravnanje i sudsko rješavanje spora različiti su postupci

Nastavni rad (1–4 min): Pitajte što rješava poravnanje, a što sud. Zatim zatražite primjer troška koji nastaje tek kada se sudionici ne mogu usuglasiti.

1.3 Pravni put postoji — koliko traje i košta?

Europska komisija u izvješću o vladavini prava za Hrvatsku iz 2025., str. 6, navodi prosječno trajanje prvostupanjskih trgovačkih parnica od 1.147 dana u 2024. To je oko 3,1 godine. Brojka opisuje tu kategoriju postupaka i godinu; nije trajanje svake parnice, međubankovnog poravnanja ili posebno sporova o digitalnim zapisima. Ne uključujemo je kao prognozu današnjeg pojedinačnog predmeta. Izvor.

Pravni postupak uključuje pristojbe i moguću naknadu za zastupanje; dodatni dokazni postupci mogu tražiti dodatne izdatke. Iznos ovisi o predmetu i radnjama, pa nema jedne poštene cijene za sve sporove. Europski portal e-pravosuđa. Uz to, organizacija troši vrijeme na dokumentaciju i upravljanje sporom.

Odvojeno računamo autorsku ilustraciju: ako je 100.000 eura zbog spora nedostupno tri godine i pretpostavimo jednostavni trošak financiranja od 5% godišnje, dobivamo 15.000 eura. To nije sudska tarifa, zakonska kamata, opaženi prosjek ni tvrdnja da je novac blokiran u svakoj parnici. Brojevi pokazuju zašto i vrijeme može biti skupo. Spor se može riješiti i dogovorom ili drugim odgovarajućim putem; ne tvrdimo da svako neslaganje završava višegodišnjom parnicom.



Figure 1.2: Hrvatska, 2024.: prosjek prvostupanjskih trgovačkih parnica

Nastavni rad (4–6 min): Odvojite na ploči izmjereni podatak 1.147 dana od pretpostavljenog izračuna 15.000 €. Pitajte gdje nastaje poslovni trošak čekanja.

1.4 Možemo li dio sporova spriječiti unaprijed?

Za naš digitalni budžet predlažemo promjenu dizajna: dopuštene radnje unaprijed preciziramo i sudionici neovisno provjeravaju prijedloge. Trebaju i zajednički postupak prihvaćanja. Sama lokalna provjera ili kopija baze nije dovoljna. Cilj je, pod izričitim pretpostavkama protokola, spriječiti da dva ispravna sudionika nespojive promjene smatraju konačno prihvaćenima.

To je istraživačko pitanje predavanja, a ne unaprijed dokaz da je blockchain jeftiniji. Dio neslaganja o digitalnom stanju mogao bi se spriječiti ili lakše provjeriti, ali sustav plaća komunikaciju, održavanje i složenije upravljanje. Pogrešna pravila, pogreške u programu i pitanja izvan digitalne evidencije ostaju mogući izvori spora. Tehničko prihvaćanje zapisa ne ukida pravne zahtjeve i sudsku zaštitu.

Pouzdana zajednička baza također može automatizirati pravila i smanjiti pogreške. Decentralizirani pristup postaje zanimljiv kada sudionici uz to ne žele jednom operatoru dati isključivu ovlast nad prihvaćenim stanjem. Tijekom predavanja uspoređujemo tu korist s cijenom i preostalim povjerenjem. Konačan izbor mora slijediti potrebe aplikacije.



Figure 1.3: Hipoteza: manje sporova o prihvaćenom stanju; trošak koordinacije ostaje

Nastavni rad (6–8 min): Tražite jedan spor koji provjera salda može spriječiti i jedan koji ostaje moguć čak uz zajednički zapis.

1.5 Što ćemo moći objasniti

Kroz cijelo predavanje pratimo isti zapis i iste sudionike. Ne tražimo da zapamtite popis protokola. Cilj je da na crtežu pokažete tko može što učiniti, koju informaciju svaki sudionik ima i koju pretpostavku vaš zaključak zahtijeva. Riječi poput „sigurno”, „dostupno” ili „decentralizirano” nisu dovoljne bez objašnjenja.

Redoslijed rada je namjeran: prvo odredimo poslovno pravilo, zatim ga pokušamo provesti na jednom mjestu, pa dodajemo kopije i ispade. Tek kad organizacije odbiju zajedničku kontrolu, prelazimo na drukčiji model povjerenja. Na kraju treba moći objasniti zašto dodavanje poslužitelja samo po sebi nije riješilo taj problem.

- Razlikovati raspodijeljenost, decentralizaciju i P2P.
- Pratiti promjene stanja kroz više replika.
- Razlikovati kašnjenje, djelomičan ispad i bizantsko ponašanje.
- Navesti pretpostavke povjerenja za predloženo rješenje.
- Obrazložiti kada je zajednički koordinator dovoljan.

Nastavni rad (8–10 min): Najavite 120 minuta uključujući kratke zadatke i raspravu. Provjerite poznaju li studenti JSON zapis i jednostavnu funkciju.

1.6 Tri organizacije, jedan digitalni budžet

Organizacije A, B i C unaprijed su poznati članovi. Svaki smije predložiti isplatu iz fonda F, ali prijedlog nije odluka. U primjeru je T1 prijenos 80 jedinica iz F u X, a T2 prijenos 60 jedinica iz F u Y. X i Y su odredišni računi u istoj digitalnoj evidenciji; oznake A, B i C označavaju sudionike i njihove čvorove, a ne odredišta isplate.

Zahtjev mora doći od ovlaštenog člana, imati pozitivan iznos i dovoljno pokrića u stanju u kojem se prihvaća. Uz stanje vodimo verziju. Ona označava korak prihvaćene povijesti, a ne vrijeme na zidnom satu. Prvo ćemo svim organizacijama dati zajedničkog koordinatora. Kasnije mijenjamo pretpostavku da mu sve vjeruju.

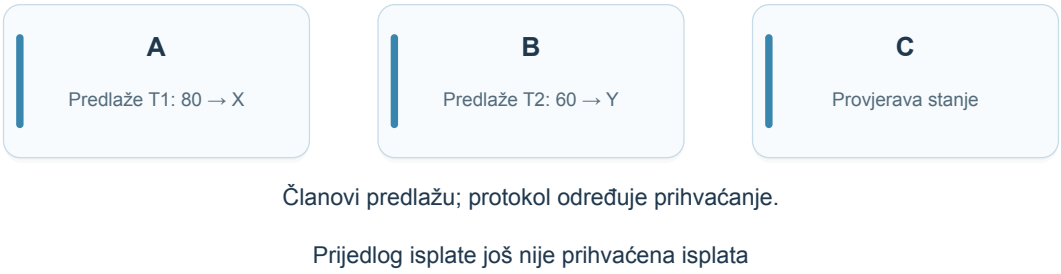


Figure 1.4: Prijedlog isplate još nije prihvaćena isplata

Nastavni rad (10–13 min): Razdvojite na ploči organizacije A/B/C od računa F/X/Y. Tko predlaže, a tko konačno prihvaća isplatu?

1.7 Stanje zapisujemo izričito

Početno stanje zapisujemo kao S7 = (F:100, X:0, Y:0, v7). T1 je prijedlog isplate 80 jedinica iz F u X uz očekivanu verziju 7. Sustav provjerava ovlast podnositelja, pozitivan iznos, pokriće i verziju. Prijedlog ne mijenja stanje samim slanjem.

Prihvaćeni T1 daje S8 = (F:20, X:80, Y:0, v8). Odbijeni zahtjev ne mijenja stanje. Obje strane prijenosa trebaju biti jedna atomarna promjena: nije dovoljno samo smanjiti F ili samo povećati X. Zbroj salda u zatvorenom primjeru ostaje 100. U nastavku dijagrami često pokazuju samo saldo F i verziju, ali krediti primatelja ostaju dio istog stanja.

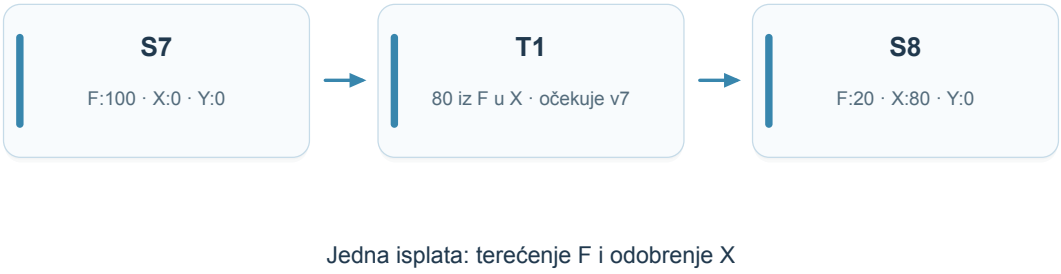


Figure 1.5: Jedna isplata: terećenje F i odobrenje X

Nastavni rad (13–16 min): Neka student izračuna sva tri salda nakon T1. Provjerite zbroj i naglasite razliku između zahtjeva i prihvaćene promjene.

1.8 Pravilo koje mora ostati istinito

Poslovno pravilo glasi: prihvaćena isplata ne smije potrošiti više jedinica nego što fond trenutačno ima. U zatvorenom modelu ne stvaramo ni uništavamo jedinice: terećenje jednog računa i odobrenje drugog čuvaju ukupni zbroj. Ovlašten podnositelj također ne smije zaobići pokriće.

Kao dodatni mehanizam naš API koristi očekivanu verziju. Iz iste verzije prihvaća najviše jednu promjenu; ostale klijent mora ponovno razmotriti nad novim stanjem. To je namjerno konzervativan mehanizam zaštite od konkurentnih upisa, a ne pravilo da su svake dvije isplate nespojive. Primjerice, 30 i 40 mogu proći redom uz osvježenu verziju. Naših 80 i 60 ne mogu proći ni u jednom redoslijedu bez novih sredstava.

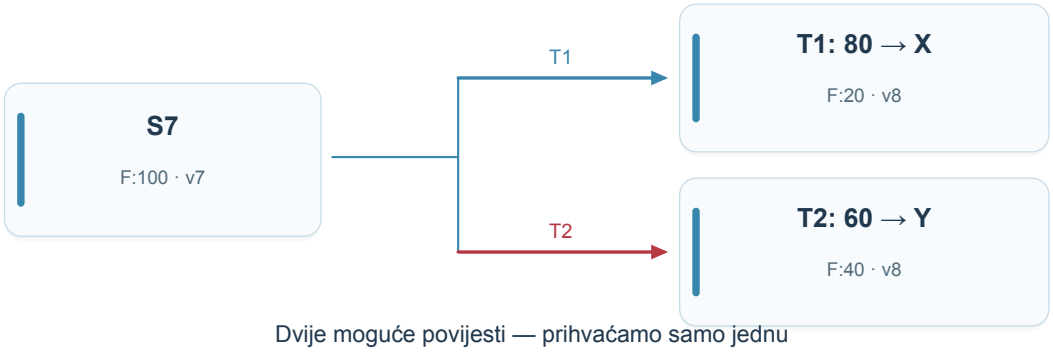


Figure 1.6: Dvije moguće povijesti — prihvaćamo samo jednu

Nastavni rad (16–19 min): Usporedite parove 80+60 i 30+40. Razlikujte nedostatak pokrića od zastarjele verzije zahtjeva.

1.9 Prvo rješenje: zajednički koordinator

Najjednostavniji početak je jedna zajednička služba s bazom podataka. Član šalje zahtjev, služba provjerava njegovu ovlast, pozitivan iznos, raspoloživi saldo i očekivanu verziju. Zatim atomarno mijenja oba računa i bilježi ishod. Ostali čitaju prihvaćeno stanje iz te službe. Zaštita od dvaju istodobnih upisa tada je lokalni problem baze i aplikacije.

Ovo rješenje nije pogrešno zato što je centralizirano. Ako organizacije vjeruju operatoru i prihvaćaju njegove ovlasti, može biti dovoljno. Treba imenovati granicu povjerenja: operator upravlja kodom, pristupom bazi i oporavkom. Klijenti ne odlučuju o konačnom stanju samo zato što na zaslonu imaju njegovu kopiju.

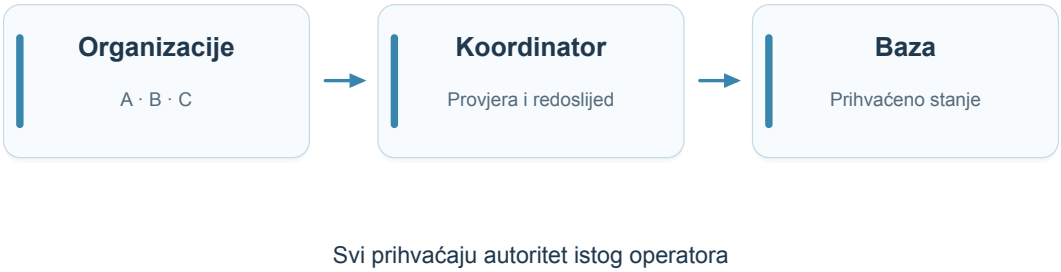


Figure 1.7: Svi prihvaćaju autoritet istog operatora

Nastavni rad (19–22 min): Povežite s BA-01, str. 40: zadržan centralizirani pristup, promijenjen primjer. Pitajte tko može promijeniti pravila.

1.10 Dva zahtjeva, jedan redoslijed

Koordinator prvo odabere T1, provjeri uvjete i atomarno prenese 80 iz F u X. Preostaje F:20, X:80, Y:0, v8. Kada provjeri T2, očekivana verzija 7 više nije aktualna. Čak i kada bi podnositelj obnovio T2 s verzijom 8, traženih 60 nema pokriće u preostalih 20. Odbijanje je normalan poslovni odgovor, ne kvar mreže.

Da je T2 proveden prvi, stanje bi bilo F:40, X:0, Y:60, v8, a T1 bi bio odbijen. Pravilo ne nalaže da baš T1 pobijedi. Nalaže dopušten redoslijed, nenegativan saldo i jednake prihvaćene promjene kod sudionika. U ovom primjeru nema dodatnih uplata između zahtjeva.

Korak	Radnja	Saldo F
0	Početak	100, v7
1	T1: isplati 80, očekuje v7	20, v8
2	T2: isplati 60, očekuje v7	odbijen; 20, v8

Nastavni rad (22–25 min): Prvo sakrijte zadnji redak i tražite predviđanje. Zatim promijenite redoslijed T1/T2 na ploči.

1.11 Naivan kod ipak može pogriješiti

Jedan operator može obrađivati zahtjeve u više dretvi. T1 pročita saldo 100 i verziju 7, a T2 pročita isto. Oba prođu provjeru. Ako zatim bez ponovne provjere provedu terećenja 80 i 60, saldo F postaje −40, X ima 80, a Y 60. Ukupni zbroj ostaje 100, ali fond je potrošio

sredstva koja nema. Ako umjesto terećenja kod prepisuje saldo iz zastarjele kopije, može nastati drukčiji problem: izgubljeni upis i nesklad salda i isplata.

Pseudokod namjerno nema zaštitu koja povezuje provjeru s promjenom. Čak i ako pomoćna funkcija atomarno prenosi iznos između računa, ranija provjera pokrića nije time zaštićena. Pozitivan iznos i ovlast jesu potrebni, ali ne rješavaju konkurentnost. Studenti trebaju pokazati točan raspored radnji koji ruši pravilo.

```
if ovlasten(clan) and iznos > 0:
    if verzija == ocekivana and saldo_F >= iznos:
        tereti_F_i_odobri_primatelja(iznos)
        verzija += 1
        return "prihvaćeno"
```

Što ako oba zahtjeva provjere saldo 100 prije prve promjene?

Nastavni rad (25–29 min): Odglumite: T1 provjeri 100, T2 provjeri 100, T1 tereti 80, T2 tereti 60. Izračunajte saldo nakon svakog koraka.

1.12 Jedna baza: atomarnost i izolacija

Ovdje govorimo o transakciji u jednoj bazi kod pouzdanog koordinatora, a ne o dogovoru neovisnih organizacija. Atomarnost osigurava da se terećenje F i odobrenje primatelja provedu zajedno ili poništе zajedno. Izolacija određuje dopušteno preplitanje konkurentnih transakcija. Sama atomarnost ne sprječava dva zahtjeva da provjere isti stari saldo.

U našem konkretnom rješenju transakcija počinje naredbom BEGIN. Dohvatimo i zaključamo redak fonda za izmjenu, primjerice SELECT … FOR UPDATE, pa provjerimo ovlast, pozitivan iznos, pokriće i verziju. Dok držimo lokot, drugi upis u isti fond čeka. Zatim ažuriramo F i račun primatelja te verziju i završavamo s COMMIT. Klijentu šaljemo potvrdu tek nakon uspješnog COMMIT-a; ako uvjet ne vrijedi ili nastane pogreška, transakciju poništavamo. Sljedeći zahtjev nakon čekanja provjerava aktualno stanje. Sve isplate moraju slijediti isti postupak.

Pretpostavljamo fiksne ovlasti članova tijekom ovog primjera i jednu bazu koja sadrži sve račune. Ažuriranje primatelja također mora izbjeći izgubljeni upis; za opće prijenose između više računa trebaju dosljedan redoslijed zaključavanja i obrada deadlocka. Primjer je konceptualni slijed, ne kompletna implementacija. Uvjetni upisi ili serijalizabilna izolacija uz obradu ponovnih pokušaja druge su mogućnosti. PostgreSQL 18: zaključavanje redaka.

To štiti poslovno pravilo unutar baze, ali ne uklanja povjerenje u njezina administratora. Ako administrator upravlja podacima i kodom, lokalna transakcija sama ne raspodjeljuje njegove ovlasti.

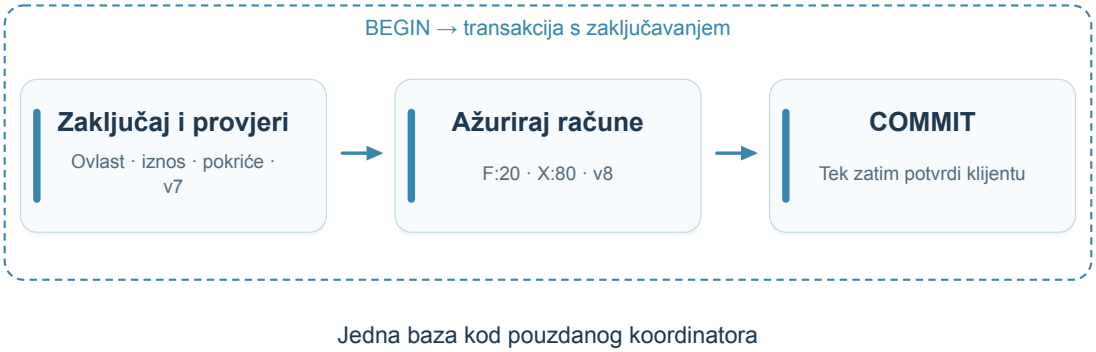


Figure 1.8: Jedna baza kod pouzdanog koordinatora

Nastavni rad (29–32 min): Neka studenti objasne dva odvojena kvara: pad između terećenja i odobrenja te dva zahtjeva koja provjere isti saldo. Povežite prvi s atomarnošću, drugi s izolacijom.

1.13 Raspodijeljenost: rad preko više procesa

Raspodijeljenost u ovom kolegiju promatramo kroz rad procesa koji razmjenjuju poruke, imaju lokalne informacije i mogu doživjeti djelomične ispade. Više procesa na istom računalu može imati neke od tih problema, ali procesi na više računala dodaju zasebne mrežne i strojne ispade. Broj fizičkih lokacija sam nije potpuna specifikacija sustava.

Na crtežu jasno razdvajamo klijenta, koordinatora i bazu. Čak je i aplikacija s jednom bazom i udaljenim klijentima raspodijeljena u komunikacijskom smislu. Kad kasnije dodamo replike baze, povećavamo raspodijeljenost pohrane; ne „pretvaramo" potpuno lokalni sustav u raspodijeljeni samo tim potezom. Kontrola i dalje može pripadati jednoj organizaciji.

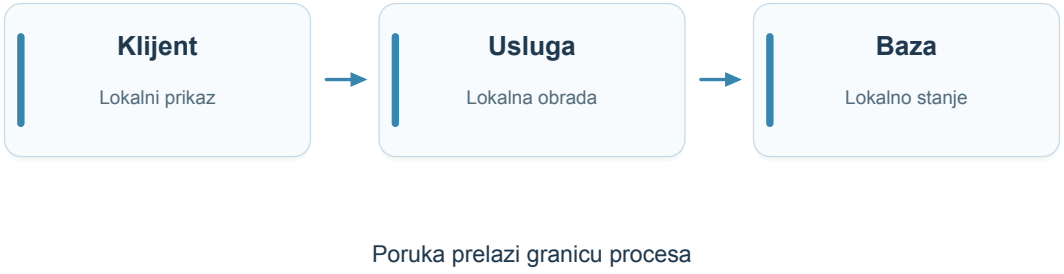


Figure 1.9: Poruka prelazi granicu procesa

Nastavni rad (32–34 min): Pitajte je li web aplikacija s jednom bazom i udaljenim korisnicima raspodijeljena. Obrazložiti granicu promatranja.

1.14 Zašto dodajemo replike?

Dosad sve isplate i sva čitanja prolaze kroz jednu bazu. Kako broj korisnika raste, prikazi salda i izvještaji mogu opteretiti primarnu bazu. Ako ona postane nedostupna, nemamo drugo mjesto spremno preuzeti njezin rad. To su razlozi za razmatranje replika: raspodjela upita za čitanje i dodatna ažurna kopija koja može pomoći oporavku.

Replika je kopija podataka koju sustav održava prijenosom promjena. A zasad jedini prihvaća upise, a B i C primjenjuju prihvaćene promjene i mogu posluživati dio čitanja. Ne tvrdimo da time povećavamo broj neovisnih upisa. Ako A padne, dostupna replika može i dalje posluživati dopuštena čitanja, ali nastavak upisa traži izbor nove primarne baze i zaštitu od istodobnog rada stare i nove primarne baze. Automatsko preuzimanje nije posljedica samog postojanja kopija.

U primjeru pretpostavljamo asinkrono slanje promjena nakon potvrde lokalnog upisa na A. Zato replika može kasniti; ako A nepovratno izgubi zapis prije slanja, preuzimanje sa zaostale replike može izgubiti potvrđenu promjenu. Politiku potvrđivanja i oporavka treba posebno odabrati. PostgreSQL 18: standby poslužitelji.

Sve kopije zasad vodi isti pouzdani operator. Poboljšavamo mogućnosti čitanja i oporavka, a ne raspodjelu upravljačkih ovlasti. Replika također nije zamjena za sigurnosnu kopiju: pogrešan upis ili brisanje može se prenijeti na sve replike. Zajednički kvar napajanja ili vjerodajnica može pogoditi više kopija. Sljedeći slajd zato pokazuje novu cijenu replikacije: različitu svježinu podataka.

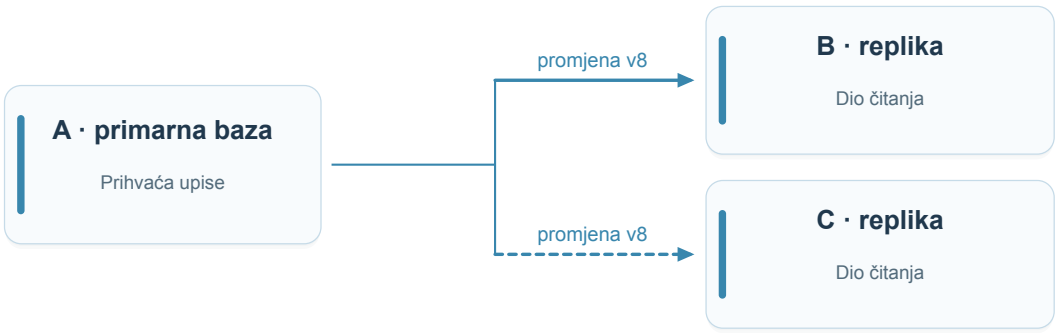


Figure 1.10: Više kopija za čitanje i oporavak; isti operator

Nastavni rad (34–37 min): Pitajte što dobivamo za čitanje, što za oporavak, a što za povjerenje u operatora. Jesu li dvije dodatne kopije dovoljne da upisi nastave nakon pada A?

1.15 Kopije nisu trenutačno jednake

Tablica prikazuje saldo fonda F i verziju na svakoj replici. U t1 A prihvaća T1: fond ima 20, a X dobiva 80. B i C još prikazuju starih 100. U t2 promjena stiže do B, a u t3 do C. Odredišni račun X ažurira se kao dio iste promjene iako zbog preglednosti nije prikazan u tablici.

To je kašnjenje širenja jedne odluke, a ne sukob dvaju donositelja odluke. Kada se sve prihvaćene promjene dostave, kopije se u ovom izvođenju izjednače. Četiri retka nisu dokaz da će svaka buduća mrežna situacija završiti tako: potrebne su pretpostavke o dostavi i oporavku. Replika koja prikazuje 100 nema time pravo neovisno odobriti novu isplatu iz tih sredstava.

Trenutak	Replika A	Replika B	Replika C
t0	100, v7	100, v7	100, v7
t1	20, v8	100, v7	100, v7
t2	20, v8	20, v8	100, v7
t3	20, v8	20, v8	20, v8

Nastavni rad (37–40 min): Za svaki redak pitajte koliko korisnik vidi na C i koliko je prihvaćeno preostalo u fondu.

1.16 Čitanje stare kopije mijenja korisnikov doživljaj

Korisnik nakon uspjeha očekuje da sljedeći prikaz sadržava njegovu promjenu. Čitanje sa zaostale replike C može prikazati 100 umjesto 20. Jedno rješenje je čitati od autoriteta za upis, drugo tražiti da replika dosegne barem verziju 8. Oba pristupa imaju trošak: čekanje, usmjeravanje ili privremenu nedostupnost čitanja.

Informativni izvještaj može dopustiti označeno kašnjenje. Konačna odluka o novoj isplati ne smije se osloniti samo na zastarjeli prikaz salda. Zato razdvajamo prikaz podataka i ovlast odlučivanja. Sučelje treba prikazati prihvaćenu isplatu i svježinu podataka bez stvaranja dojma da je stari saldo ponovno raspoloživ.



Figure 1.11: Staro čitanje nije dokaz poništene isplate

Nastavni rad (40–42 min): Tražite jedno čitanje gdje je kašnjenje prihvatljivo i jedno gdje bi bilo opasno.

1.17 Zahtjev je izvršen, potvrda nije stigla

U ovom izvođenju zahtjev stigne do koordinatora. Koordinator provede T1: F ima 20, X ima 80, verzija je 8 i šalje uspješan odgovor. Odgovor se izgubi ili toliko kasni da klijent prestane čekati. Klijent ne smije zaključiti da upisa nije bilo. Iz njegove perspektive isti timeout moguć je i kada zahtjev uopće nije stigao.

Ta neizvjesnost nastaje na granici sustava. Waldo i suradnici u odjeljku 4.3 opisuju djelomične ispade kao problem koji udaljeni poziv razlikuje od običnog lokalnog poziva: dio sustava može nastaviti raditi dok drugi dio ne odgovara. Za našu aplikaciju posljedica je konkretna: sučelje treba stanje „ishod nepoznat”, a ne izmišljenu konačnu poruku. A Note on Distributed Computing, odjeljak 4.3.

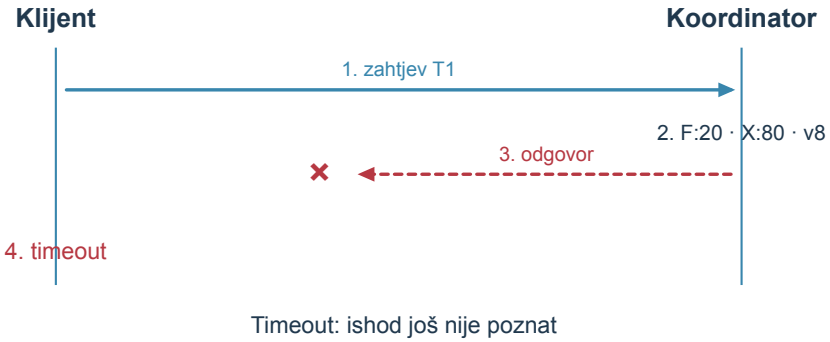


Figure 1.12: Timeout: ishod još nije poznat

Nastavni rad (42–45 min): Usporedite dva izvođenja s istim zaslonom klijenta. Pitajte koja činjenica nedostaje.

1.18 Ponovljeni pokušaj treba isti identitet

Zahtjevu dodajemo identifikator K42. Koordinator uz prihvaćenu isplatu bilježi rezultat vezan uz klijenta i taj identifikator. Ako isti klijent ponovi isti zahtjev, može dobiti raniji rezultat umjesto još jedne promjene. ID pritom ne smije biti ponovno iskorišten za drukčiji sadržaj: takav slučaj treba odbiti ili posebno obraditi.

Pohrana rezultata i promjena stanja moraju biti povezane tako da pad između njih ne stvori novu dvosmislenost. U nastavnom modelu pretpostavljamo atomarnu pohranu obaju zapisa i čuvanje ID-a tijekom promatranog razdoblja. Time ne tvrdimo da je mreža dostavila poruku točno jednom. Ograničili smo učinak ponavljanja na strani usluge. AWS Builders' Library opisuje taj obrazac.

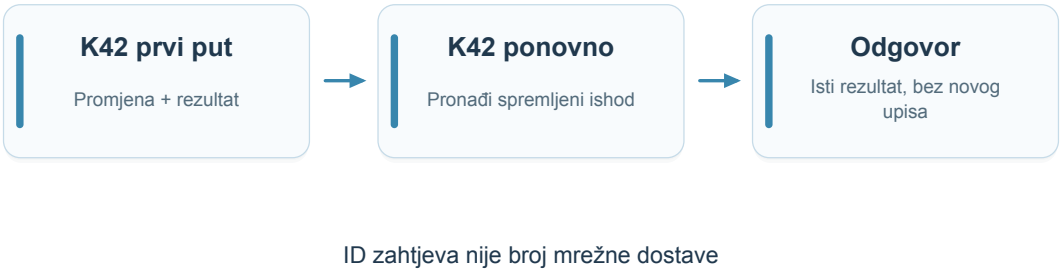


Figure 1.13: ID zahtjeva nije broj mrežne dostave

Nastavni rad (45–48 min): Pitajte što ako klijent za ponovni pokušaj stvori novi ID. Razlikujte ponovni pokušaj od nove korisničke namjere.

1.19 Što znači djelomičan ispad?

Čvor A može normalno posluživati lokalne korisnike, dok čvor B do njega ne može doći. Čvor C možda može komunicirati s oboma. „Sustav radi" zato nije dovoljno precizan opis: treba navesti tko koju operaciju može izvršiti. Za razliku od jedne funkcije u našem procesu, udaljena usluga može nastaviti mijenjati stanje i nakon što smo prestali čekati odgovor.

Izvor s Harvarda koristimo za temeljnu razliku, ali ne prenosimo njegove povijesne tvrdnje o tadašnjim proizvodima kao današnja ograničenja. Naš konkretni primjer pokazuje zašto i API i korisničko sučelje trebaju izraziti nepoznat ishod. Poboljšanje samo jedne komponente ne uklanja svaku neizvjesnost na komunikacijskoj granici.

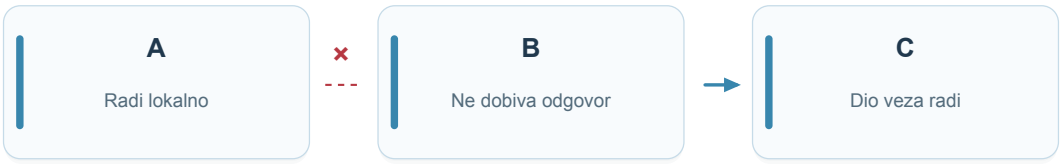


Figure 1.14: Dostupnost promatramo za određenog sudionika i radnju

Nastavni rad (48–50 min): Pročitajte status iz perspektive A, zatim B. Isti raspored može izgledati drukčije različitim korisnicima.

1.20 Prije jamstva: napišimo model

Model sustava navodi što dopuštamo okolini i kvarovima. Bez njega je tvrdnja „radi i kad nešto padne" neprovjerljiva. Za sada imamo tri poznata sudionika, ne proizvoljan broj anonimnih identiteta. Za promatrano izvršavanje nemamo poznatu gornju granicu dostave, a pošteni čvorovi slijede pravila dok rade. Zaustavljen čvor ne odgovara; ne šalje namjerno lažan podatak.

To ne znači da stvarne mreže uvijek kasne neograničeno. Znači da naš zaključak ne smije ovisiti o proizvoljno odabranom roku kao dokazu pada. Ako kasnije dodamo pretpostavku o oporavku ili o poznatoj granici, moramo je izričito navesti. Otpornost na ispad pohrane također nije dana besplatno; u ovom primjeru je posebno pretpostavljena.

Pretpostavka	**Za naš sljedeći primjer**
Članovi	A, B i C, unaprijed poznati
Poruke	mogu kasniti, biti ponovljene ili izgubljene
Vrijeme	nemamo poznat konačan rok dostave
Čvorovi	zasad mogu stati; ne izmišljaju poruke
Pohrana	trajnost lokalnog prihvaćenog zapisa pretpostavljena

Nastavni rad (50–53 min): Neka studenti dodaju jedan rizik koji model zasad ne pokriva. Očekivano: zlonamjeran administrator ili gubitak trajne pohrane.

1.21 Spor čvor i zaustavljen čvor mogu izgledati isto

Pretpostavimo da B nije dobio odgovor od A tri sekunde. Jedno moguće izvođenje jest da je A ugašen. Drugo je da A radi, ali odgovor čeka u mreži. Treće je da je A završio posao i stao prije slanja odgovora. Iz samog izostanka poruke B ne zna koje se izvođenje dogodilo.

Timeout i dalje ima praktičnu vrijednost: omogućuje prestanak čekanja i pokretanje oporavka. No odluka da drugi čvor preuzme upise mora uzeti u obzir mogućnost da prvi i dalje radi. Kad bismo obojici dali neograničeno pravo konačnog potvrđivanja, mogli bismo dobiti dvije povijesti. Ovdje prepoznajemo problem; ne uvodimo još protokol izbora vođe.

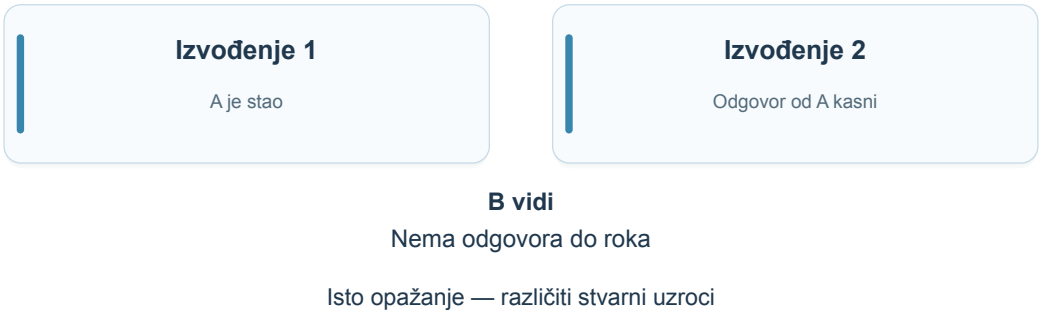


Figure 1.15: Isto opažanje — različiti stvarni uzroci

Nastavni rad (53–56 min): Tražite isti opaženi trag za spori i ugašeni čvor. Ne tvrditi da se kvar nikada ne može otkriti uz dodatne pretpostavke.

1.22 Mreža se razdvojila

Mrežna particija u primjeru znači da poruke između skupine A–B i C ne prolaze. To ne zahtijeva gašenje svih računala. Obje strane mogu primati lokalne zahtjeve i imati podatke koji su bili jednaki neposredno prije razdvajanja. Ako nastave odlučivati svaka za sebe, više nema zajedničkog mjesta koje odmah vidi oba zahtjeva.

Važno je nacrtati konkretne nedostupne veze. Rečenica „internet je pao” skriva tko još s kim razgovara. Naš crtež koristi dva dijela radi jednostavnosti; stvarni rasporedi mogu biti složeniji. Za daljnji zadatak obje strane polaze iz F:100, v7 i dobivaju zahtjeve za isplate 80 i 60.



Figure 1.16: Kopije nastavljaju rad, komunikacija je prekinuta

Nastavni rad (56–59 min): Na ploči precrtajte veze prema C, ali ostavite korisnika kod C. Pitajte može li C još prikazivati lokalne podatke.

1.23 Dvije strane ne smiju obećati suprotno

Prije razdvajanja obje strane imaju F:100, X:0, Y:0, v7. A–B konačno potvrde T1 pa vide F:20, X:80, Y:0. C konačno potvrdi T2 pa vidi F:40, X:0, Y:60. Kada se veza vrati, ne možemo obje isplate uključiti u valjanu povijest iz početnih 100 bez novih sredstava. Njihovim zbrajanjem F bi pao na −40; odbacivanjem jedne morali bismo povući već izdanu konačnu potvrdu.

Odvojena strana može primati zahtjeve kao nepotvrđene prijedloge ili privremeno zaustaviti isplate. Drukčiji dizajn može unaprijed podijeliti prava potrošnje između strana, ali tada svaka više ne raspolaže cijelih 100. To nije pretpostavka našeg primjera. Pokazujemo zašto neovisne odluke nad istim raspoloživim sredstvima krše odabrano pravilo.



Figure 1.17: Spajanje kopija ne stvara dodatnih 40 jedinica

Nastavni rad (59–63 min): Rasprava: što smije pisati na gumbu i u potvrdi dok je zahtjev samo prijedlog?

1.24 Tri različita pitanja o arhitekturi

Raspodijeljenost opisuje razmještaj procesa, stanja i komunikacije. Decentralizaciju ovdje promatramo kao raspodjelu relevantnih ovlasti između neovisnih sudionika. P2P opisuje uloge u razmjeni usluga: sudionici nisu samo potrošači nego i pružatelji resursa. To su radne razlike za analizu, ne tvrdnja da postoji jedna univerzalna brojčana mjera decentralizacije.

RFC 5694 upozorava da P2P i centralizirani elementi mogu postojati u istom sustavu. Zato iz mreže vršnjaka ne zaključujemo automatski da nitko nema središnju kontrolu. Može postojati središnji upis članova ili jedini ključ za nadogradnju. Za svaki sustav pitamo koja funkcija ima koju strukturu. RFC 5694, odjeljci 2–4.

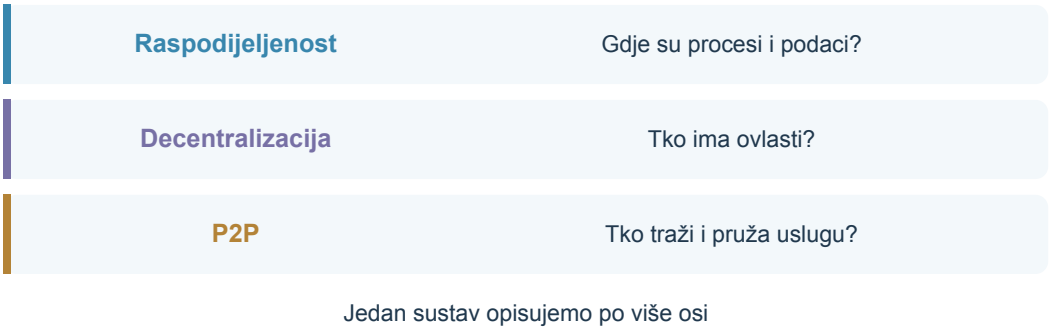


Figure 1.18: Jedan sustav opisujemo po više osi

Nastavni rad (63–66 min): Neka studenti za svaku riječ postave pitanje, umjesto da pokušaju složiti ljestvicu „više je bolje”.

1.25 Sto poslužitelja, jedan administrator

Ako jedna organizacija upravlja sa sto replika, raspodijelila je izvršavanje i možda povećala otpornost na pojedine kvarove. Time nije nužno raspodijelila ovlast promjene pravila. Jedan administratorski račun, zajednička nadogradnja ili odluka operatora može utjecati na sve kopije.

Za zajednički budžet zajednički operator može biti razumno ugovorno rješenje. Ako je problem samo dostupnost, dodatne replike mogu pomoći. Ako je problem da operator smije neprimjetno prepisati povijest, sto njegovih replika ne uklanja taj izvor povjerenja. Pitanje neovisnosti odnosi se na kontrolu relevantnu za prijetnju koju analiziramo, a ne samo na broj kutija na dijagramu.

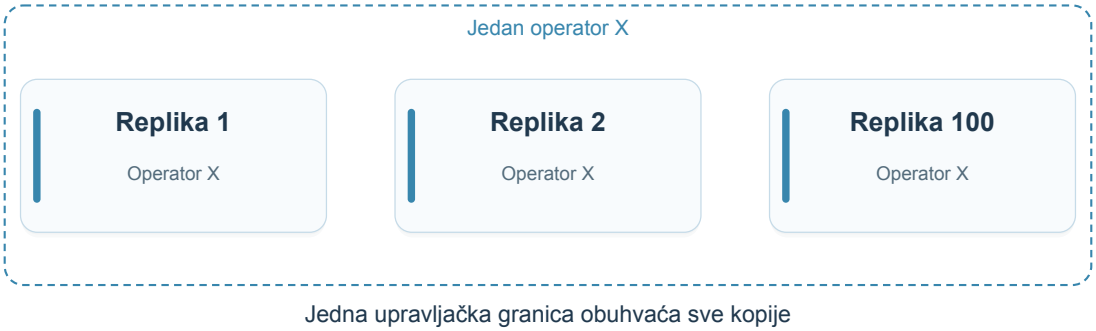


Figure 1.19: Jedna upravljačka granica obuhvaća sve kopije

Nastavni rad (66–69 min): Prekrijte sve čvorove jednom granicom „jedan administrator". Pitajte što ostaje neriješeno.

1.26 P2P opisuje razmjenu, ne jamči dogovor

U P2P dijelu aplikacije A može tražiti zapis od B, a zatim drugom sudioniku pružiti podatak koji ima. To razlikujemo od stroge podjele na klijente koji samo traže i poslužitelje koji samo pružaju uslugu. Ne zahtijevamo da svaki čvor bude povezan sa svakim drugim niti da svi imaju jednake resurse.

Ni P2P razmjena ni topologija trokuta ne definiraju pravilo prihvaćanja. A i B mogu vrlo učinkovito razmijeniti dvije suprotne tvrdnje. Treba dodatno odrediti članstvo, ovlasti, provjeru i dogovor. Primjer je nastavna primjena razlike iz RFC-a, a ne tvrdnja o određenom današnjem proizvodu.

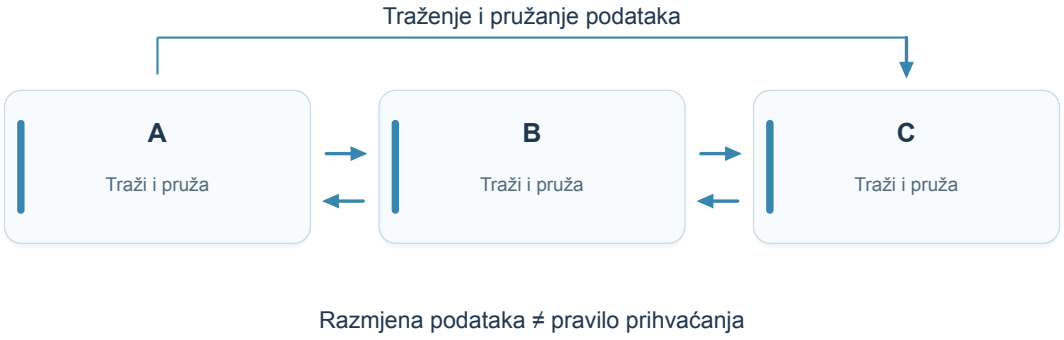


Figure 1.20: Razmjena podataka pravilo prihvaćanja

Nastavni rad (69–71 min): Nacrtajte strelice u oba smjera i zatražite da student navede informaciju koja još nedostaje.

1.27 Razvrstajte tri sustava

Zadatak traži obrazloženje, ne samo naljepnicu. Prvi sustav ima raspodijeljenu pohranu, ali opisana kontrola ostaje kod tvrtke. Drugi raspodjeljuje odobravanje između organizacija, a tehničko izvršavanje i dostupnost još ovise o operatoru. Treći ima P2P razmjenu datoteka i središnju ovlast za članstvo.

Za neke osi tekst nema dovoljno podataka. Primjerice, iz činjenice da tri organizacije odobravaju promjene ne znamo jesu li te ovlasti tehnički provedive ili samo ugovorne. Dobar odgovor navodi što znamo, što ne znamo i koje bismo dodatno pitanje postavili. Time vježbamo disciplinu važniju od prepoznavanja buzzworda.

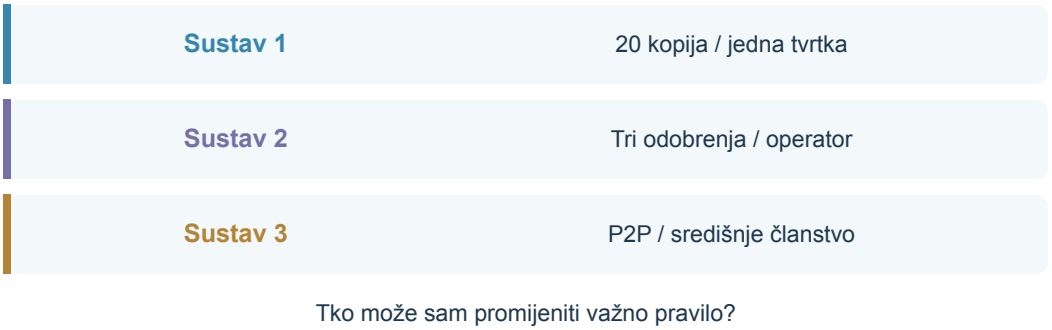


Figure 1.21: Tko može sam promijeniti važno pravilo?

Nastavni rad (71–74 min): Dvije minute u parovima, zatim po jedan zaključak za svaki sustav. Priznati „nedovoljno informacija” uz dobro dodatno pitanje.

1.28 Nova pretpostavka: nema zajedničkog operatora

Sada mijenjamo zahtjev, a ne samo tehnički raspored. Organizacije ne žele dati jednoj organizaciji ovlast konačnog odlučivanja. Svaki drži svoju kopiju i provodi dogovorena pravila. Neovisnost upravljanja donosi novu zadaću: treba se usuglasiti ne samo oko sadržaja poruke nego oko toga koja je promjena prihvaćena.

Mreža i dalje može kasniti, a čvorovi i dalje mogu stati. Uz to, različiti operatori mogu imati različite interese. To ne znači da svi moraju biti zlonamjerni. Model mora objasniti što se događa ako se netko ne pridržava protokola. Decentralizacija ovdje premješta dio povjerenja iz jedne organizacije u pravila i pretpostavke zajedničkog postupka.



Figure 1.22: Dogovor mora povezati neovisne operatore

Nastavni rad (74–77 min): Pitajte koja se pretpostavka promijenila u odnosu na slajd sa sto poslužitelja. Nije se samo povećao broj strojeva.

1.29 Povjerenje napišemo kao konkretne ovlasti

„Ne vjerujemo A-u" preširoka je izjava. Možemo vjerovati A-u da ispravno identificira svoje zaposlenike, ali mu ne dati pravo da sam briše tuđe zapise. Možemo dopustiti operatoru slanje poruka, ali od njega ne preuzimati konačnu odluku bez provjere. Treba opisati ovlasti i posljedice njihove zlouporabe.

Za budžet sastavljamo malu matricu: tko podnosi zahtjev, tko provjerava lokalne uvjete i tko potvrđuje prihvaćanje. Zasebno razmatramo promjenu popisa članova i nadogradnju koda. Ako ta dva koraka ostanu pod jednim ključem, ondje je važna središnja ovlast bez obzira na način razmjene podataka.

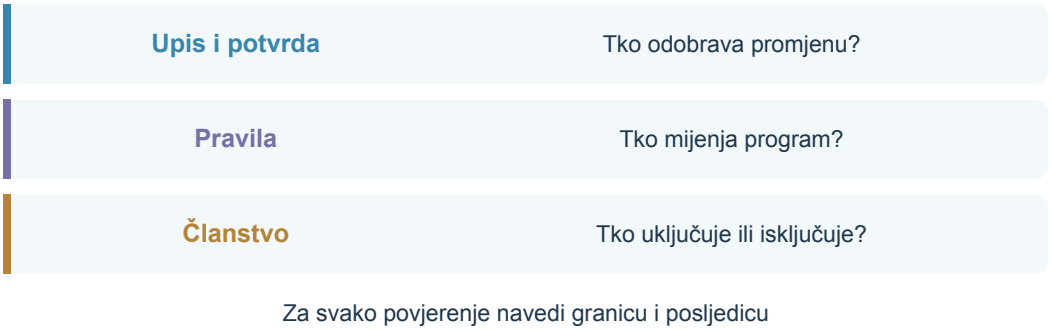


Figure 1.23: Za svako povjerenje navedi granicu i posljedicu

Nastavni rad (77–80 min): Neka studenti za svaku ovlast navedu jedan rizik i osobu/organizaciju kojoj bi je dali.

1.30 Pad i proizvoljno ponašanje nisu isti kvar

U modelu pada čvor prestane izvršavati korake; ostatak sustava mora raditi s manjkom odgovora. Bizantski model dopušta proizvoljno odstupanje od protokola, uključujući selektivno slanje, lažne vrijednosti i međusobno proturječne tvrdnje. Uzrok može biti napad, programska pogreška ili kvar; za ponašanje protokola važna je dopuštena radnja, ne moralna ocjena operatora.

Lamport, Shostak i Pease uvode problem pouzdanog dogovora kad neispravne komponente različitim dijelovima sustava daju proturječne informacije. Ovdje uzimamo tu razliku, a njihove uvjete i algoritme ostavljamo za sljedeće predavanje. Ne zaključujemo prag otpornosti bez specifikacije komunikacije, identiteta i protokola. The Byzantine Generals Problem, uvod.

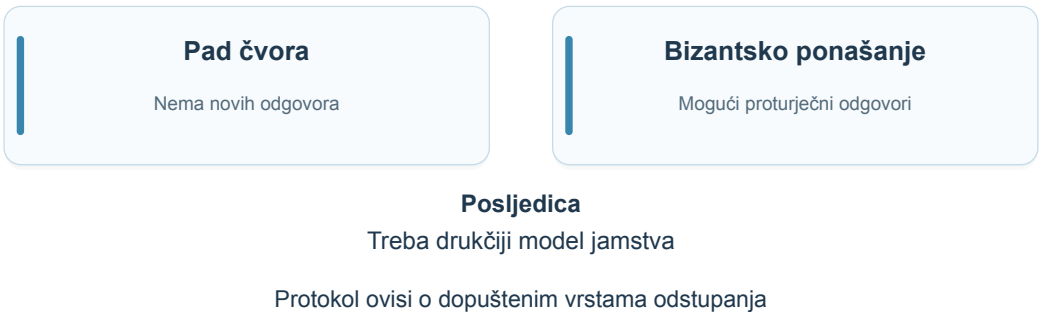


Figure 1.24: Protokol ovisi o dopuštenim vrstama odstupanja

Nastavni rad (80–83 min): Pitajte mora li bizantski kvar biti namjerna prijevara. Ne mora.

1.31 Isti pošiljatelj govori dvije stvari

A odstupa od pravila i šalje nespojive potvrde. B vidi da je prihvaćen T1: 80 za X, a C da je prihvaćen T2: 60 za Y. Oba mogu prepoznati A kao pošiljatelja. Ipak, obje isplate ne stanu u početni budžet od 100. Čak i kada A navede pojedinačno ispravan novi saldo za svaki zahtjev, ukupna obećanja nemaju pokriće.

Potrebna je dodatna razmjena informacija i pravilo zajedničkog prihvaćanja. Informacija „A je poslao” nije dovoljna. Kod zaostale replike postoji jedna prihvaćena odluka i različita svježina znanja. Ovdje postoje nespojive tvrdnje o tome što je konačno prihvaćeno.

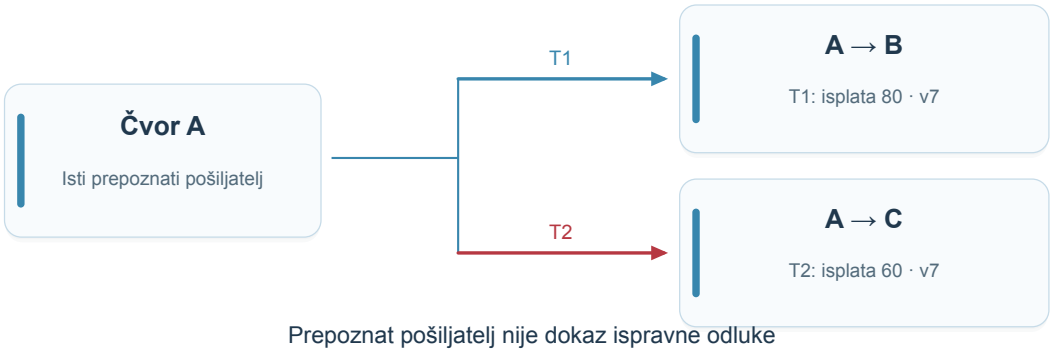


Figure 1.25: Prepoznat pošiljatelj nije dokaz ispravne odluke

Nastavni rad (83–86 min): Na ploči razdvojite pogled B-a i C-a. Što svaki od njih može zaključiti bez dodatne poruke?

1.32 Kašnjenje ili proturječje?

U prvom slučaju opažanje je kompatibilno s poštenom, zaostalom replikom. Samo iz stare verzije ne zaključujemo da C laže. U drugom slučaju imamo konkretne proturječne potvrde koje krše pravilo, ako vjerodostojno znamo njihovo podrijetlo i značenje. Ne mora svaka razlika u zaslonima biti dokaz istog problema.

Važno je odrediti koji dokaz imamo. „Moj ekran pokazuje drugo" slabije je od dvije poruke koje potvrđuju dvije nespojive konačne isplate iz iste verzije. Čak ni detekcija proturječja ne određuje automatski koja promjena vrijedi. To treba urediti protokol i način oporavka. Primjer priprema studente da pri analizi ispada ne zamijene opažanje za objašnjenje.

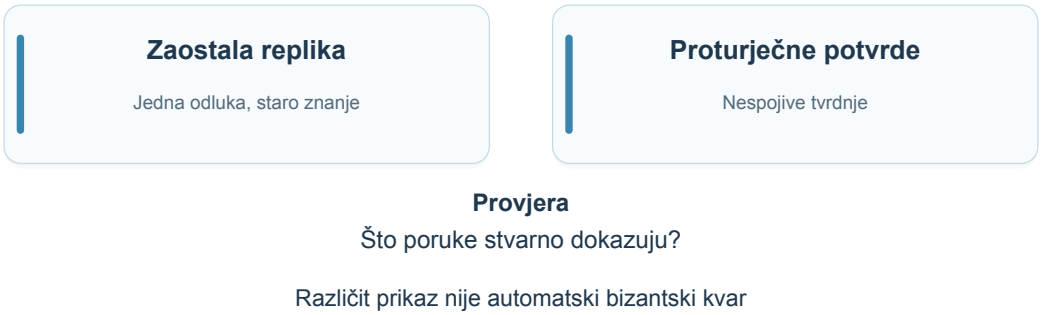


Figure 1.26: Različit prikaz nije automatski bizantski kvar

Nastavni rad (86–89 min): Neka studenti klasificiraju oba slučaja i navedu što bi još zatražili za dijagnozu.

1.33 Identitet ne zamjenjuje pravilo prihvaćanja

Pretpostavimo samo da možemo provjeriti od kojeg je člana stigla poruka; kriptografski mehanizam još ne razrađujemo. Provjeren identitet ne daje sadržaju automatsku valjanost. Poznati član može poslati zahtjev za isplatu s verzijom koja više nije aktualna, tražiti radnju za koju nije ovlašten ili dati suprotne odgovore različitim primateljima.

Provjera zato ima više slojeva: podrijetlo, dopuštenost radnje prema stanju i dogovor da je promjena prihvaćena. Ova razlika štiti od česte prečice da je svaka autentificirana poruka ujedno zajednička odluka. Mehanizme potpisa obradit ćemo kad budu potrebni za provjerljive zapise.

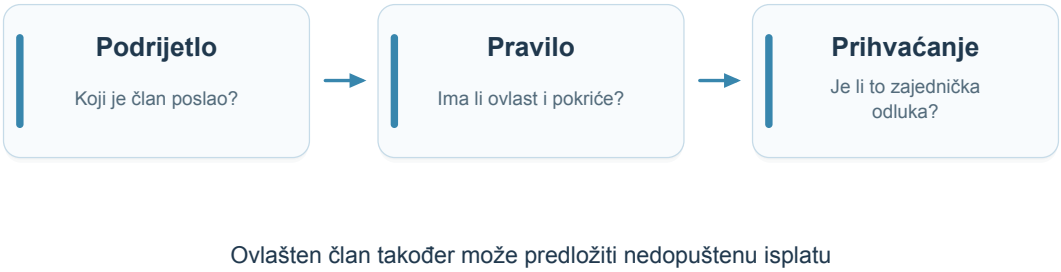


Figure 1.27: Ovlašten član također može predložiti nedopuštenu isplatu

Nastavni rad (89–91 min): Nemojte uvoditi hash funkcije ni sintaksu potpisa. Studenti trebaju samo prepoznati tri različita pitanja.

1.34 „Uzmimo dva od tri” još nije protokol

Neka B prihvati T1 kad vidi dvije potvrde, svoju i A-ovu. C prema istom pravilu prihvati T2 jer vidi svoju i A-ovu potvrdu. A je potvrdio oba zahtjeva. Obje skupine imaju po dva od tri člana, ali jedini član u njihovu presjeku odstupa od protokola. Zato pravilo „većina glasova” u ovom naivnom primjeru nije osiguralo jedinstven rezultat.

Ovo nije dokaz da svaki dogovor s tri sudionika uvijek mora propasti, niti izvod općeg praga tolerancije. To je protuprimjer točno navedenom postupku. Za jamstvo treba opisati tko i kada smije glasati, koje poruke razmjenjuje, kakve kvarove toleriramo i pod kojim uvjetima čekanje završava. Tu ćemo u predavanju 02 formalizirati problem konsenzusa.

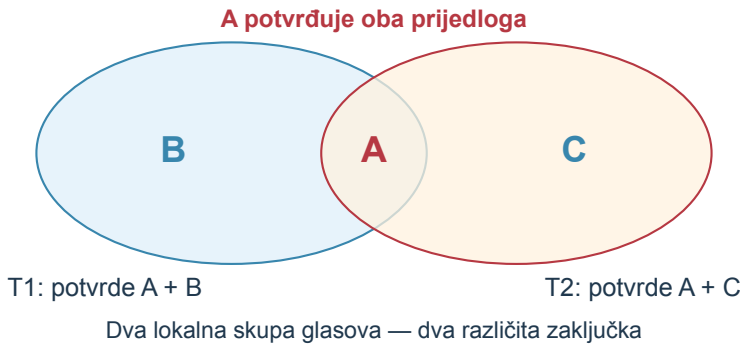


Figure 1.28: Dva lokalna skupa glasova — dva različita zaključka

Nastavni rad (91–95 min): Nacrtajte skupove {A,B} i {A,C}. Tražite da student pokaže zašto sam presjek nije dovoljan.

1.35 Tko uopće ima pravo glasa?

Naš primjer cijelo vrijeme ima dogovoreno članstvo. Ako primimo deset poruka od A, to nisu glasovi deset organizacija. Jednako tako deset procesa kojima upravlja A ne znači deset neovisnih članova. Pravilo glasovanja mora vezati glas uz određeni identitet i definirati promjene tog skupa.

Otvoreno pristupanje postavlja dodatno pitanje: kako spriječiti da jedna strana stvori proizvoljan broj identiteta i tako promijeni ishod? Danas to samo izdvajamo kao novo ograničenje. Ne miješamo model bizantskog ponašanja poznatih članova s otpornošću na stvaranje identiteta. Precizno članstvo jedna je od stavki s kojom završavamo specifikaciju za sljedeće predavanje.

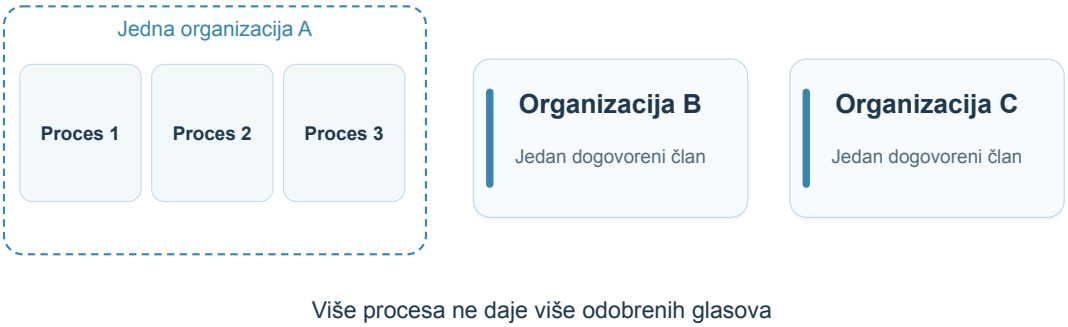


Figure 1.29: Više procesa ne daje više odobrenih glasova

Nastavni rad (95–97 min): Pitajte što je glas u našem modelu: paket, proces, račun ili organizacija? Odgovor ovisi o dogovorenom članstvu.

1.36 Radionica: dvije isplate iz istog budžeta

Radionicu provodimo na papiru. Svaki sudionik ima stupce primljene poruke, lokalno stanje i konačne potvrde. Prvo izvedite poštenog koordinatora koji odabere T1 i odbije T2. Potom ponovite s izgubljenom potvrdom B-u. Treći put koristite naivno pravilo dva glasa, pri čemu A šalje različite potvrde B-u i C-u. Nemojte potajno usklađivati zapise razgovorom izvan zadanih poruka.

Cilj nije osmisliti gotov otporan protokol u šest minuta. Cilj je proizvesti trag iz kojeg se vidi gdje je izgubljeno jamstvo. Ako skupina izabere čekanje, treba jasno reći na koju informaciju čeka i što se događa ako ona ne stigne. Čekanje može zaštititi pravilo, ali ne jamči dovršetak.

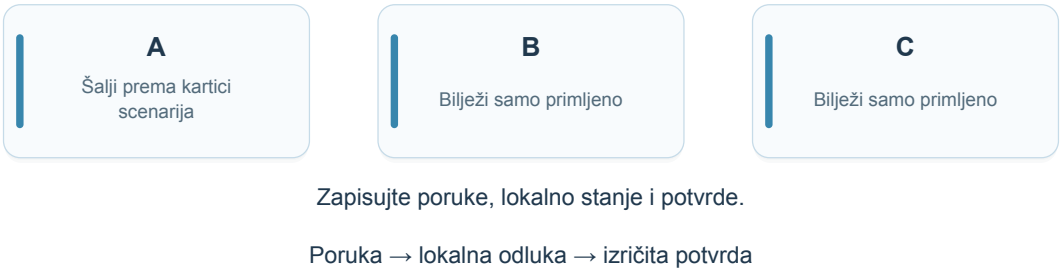


Figure 1.30: Poruka → lokalna odluka → izričita potvrda

Nastavni rad (97–103 min): 1 min raspodjela uloga; 3 min izvođenje; 2 min bilježenje. Nastavnik glumi mrežu i zadržava jednu poruku. Ne tražiti stvarnu mrežu ni alat.

1.37 Analiza: gdje se izgubilo jamstvo?

U prvom izvođenju postoji jedan ovlašteni donositelj odluke i jedna prihvaćena isplata. U drugom stanje može biti ispravno, a znanje klijenta nepotpuno. U trećem naivno pravilo dopušta dvije konačne odluke. Te razlike treba imenovati prije prijedloga popravka. „Dodaj još jednu kopiju” ne objašnjava kako će se promijeniti pravilo prihvaćanja.

Za svaki prijedlog popravka tražimo preostalu slabost. Ako svi čekaju sve, što radi zaustavljen član? Ako netko odlučuje sam, kome je povjereno upravljanje? Ako glasujemo, koje članstvo i dopuštene kvarove pretpostavljamo? Odgovori pokazuju zašto je dogovor o redoslijedu poseban problem, a ne dodatna strelica na arhitekturnom crtežu.

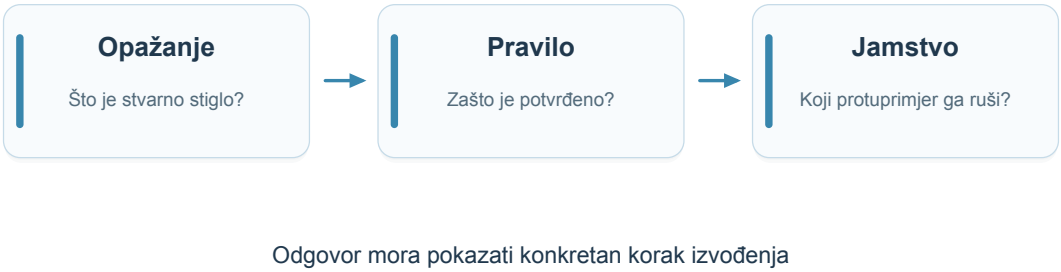


Figure 1.31: Odgovor mora pokazati konkretan korak izvođenja

Nastavni rad (103–107 min): Uzeti dva traga iz publike. Pohvaliti konkretan protuprimjer, ne složenost predloženog protokola.

1.38 Odaberite arhitekturu prema problemu

Uspoređujemo potrebe, ne dodjeljujemo moralne ocjene arhitekturama. Zajednička baza ima manje sudionika u odlučivanju i može biti dobar odgovor kad je operator prihvaćen. Replikacija rješava dio problema rada s kopijama i oporavka, ali traži pravilo potvrđivanja i preuzimanja. Neovisno odlučivanje više organizacija uvodi dodatnu razmjenu poruka, složenije kvarove i pitanje zajedničke nadogradnje.

Za budžet pitamo treba li svaka organizacija tehnički provjeravati odluke ili je dovoljan ugovor i revizija rada operatora. Ne postoji obveza koristiti *blockchain* samo zato što je tema kolegija. Danas još nemamo njegov mehanizam; imamo jasniji problem koji bi odabrana tehnologija trebala opravdano riješiti.

Potreba	**Početni smjer**
Svi prihvaćaju jednog operatora	zajednička služba i baza
Treba preživjeti pad pojedine kopije	replikacija i plan oporavka
Neovisni operatori moraju provjeravati odluke	zajednički protokol i model kvarova

Više koordinacije ima cijenu.

Nastavni rad (107–110 min): Za naš budžet tražite dva različita, ali obrazložena izbora. Dodatne pretpostavke zapišite uz izbor.

1.39 Provjerite uvjerljiv AI prijedlog

Ovo je namjerno sastavljen nastavnikov primjer lošeg prijedloga, ne izlaz određenog AI proizvoda. Tri kopije ne govore tko ih kontrolira. Prva potvrda ne dokazuje što ostale kopije prihvaćaju niti štiti od proturječnih potvrda. Novi zahtjev nakon timeouta može predstavljati novu namjeru umjesto ponavljanja stare i proizvesti neželjen dodatni učinak.

Dobar pregled koda počinje protuprimjerom. Zatražite izgubljeni odgovor nakon upisa, dvije potvrde iz iste verzije i tri poslužitelja pod jednim administratorskim računom. Zatim tražite popravljenu specifikaciju: stabilan identitet pokušaja, jasno pravilo potvrđivanja i odvojenu tvrdnju o kontroli. Ni ta poboljšanja sama ne čine cjelovit protokol tolerantan na bizantske kvarove.

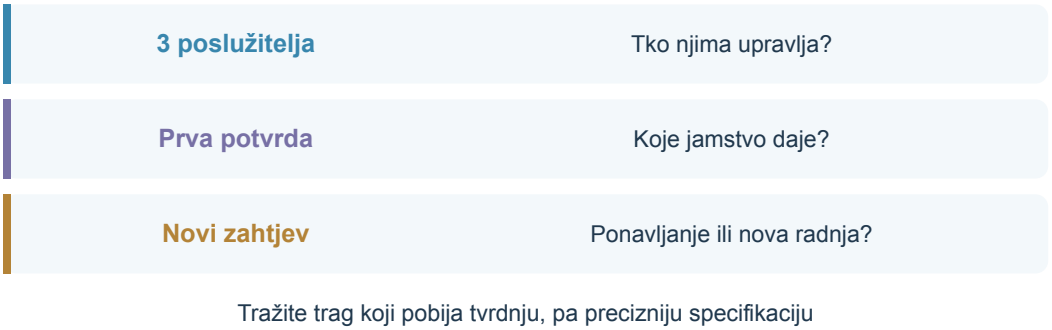


Figure 1.32: Tražite trag koji pobija tvrdnju, pa precizniju specifikaciju

Nastavni rad (110–114 min): 2 min u parovima + 2 min rasprava. Očekivani nalazi: kontrola, konačnost prve potvrde, nepoznat ishod i novi ID.

1.40 Sažetak: tri razlike koje ostaju

Predavanje je krenulo od jednog zapisa i poslovnog pravila. Lokalna atomarna promjena riješila je sukob u jednoj bazi. Replike su otvorile pitanje svježine, a mreža neizvjesnost o ishodu. Neovisni operatori promijenili su granicu povjerenja i dodali mogućnost proturječnih poruka. Svaki korak zahtijevao je novu, izričitu pretpostavku.

Provjerite možete li objasniti razliku između raspodijeljenosti, decentralizacije i P2P-a bez navođenja imena proizvoda. Zatim objasnite zašto su zaostala kopija i dvije suprotne konačne potvrde različiti problemi. Konačno, navedite što u našem primjeru znamo o članovima i vremenu dostave. To je podloga na koju će se nasloniti precizni zahtjevi konsenzusa.

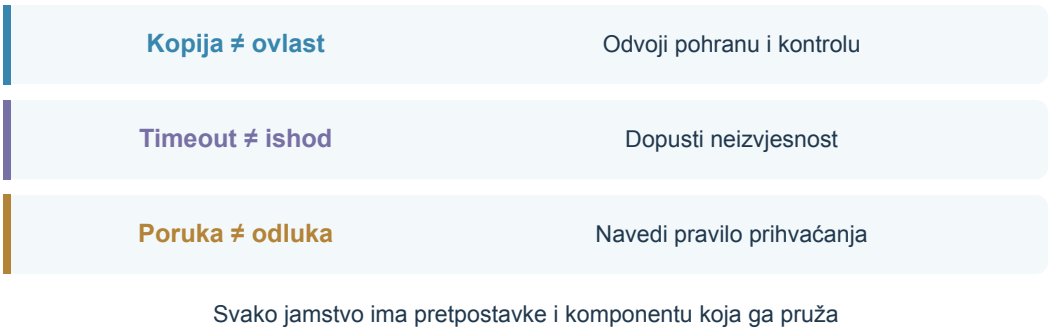


Figure 1.33: Svako jamstvo ima pretpostavke i komponentu koja ga pruža

Nastavni rad (114–116 min): Brzi izlazni odgovori: jedno pitanje po redu učionice, bez ponovnog čitanja definicija.

1.41 Sljedeći izazov: dogovor bez zajedničkog šefa

Za sljedeće predavanje predajemo specifikaciju, a ne gotov odgovor: postoje poznati sudionici, dopuštena odstupanja i zahtjev da ispravni sudionici ne donose nespojive konačne odluke. Treba još precizno odrediti koje su odluke valjane i pod kojim uvjetima postupak mora završiti. Ako zaboravimo bilo koji od tih dijelova, lako je predložiti prividno rješenje koje uvijek čeka ili uvijek prihvaća unaprijed zadanu vrijednost.

Za pripremu nacrtajte dva prijedloga: jedan koji čeka sve odgovore i jedan koji odlučuje nakon prvih nekoliko. Za svaki napišite primjer u kojem ima problem. Ne traži se naziv algoritma niti programska implementacija. Traži se veza između uvjeta rada i jamstva koje tvrdite da pružate.



Figure 1.34: Dobra specifikacija prethodi izboru protokola

Nastavni rad (116–119 min): Ostavite minute za završna pitanja. Najavite konsenzus i bizantske generale u 02, bez preuzimanja sljedećeg predavanja.

1.42 Literatura za produbljivanje

Za prvo čitanje odaberite Waldoov odjeljak o djelomičnim ispadima i usporedite ga s primjerom izgubljene potvrde. Iz RFC-a izdvojite jednu funkciju sustava koja može biti središnja čak kad je razmjena P2P. U uvodu rada o bizantskim generalima prepoznajte problem različitih informacija poslanih različitim primateljima. Algoritamski dio tog rada nije preduvjet za današnju vježbu.

AWS-ov tekst povezuje ponavljanje zahtjeva s API dizajnom. Primjeri i dijagrami našeg digitalnog budžeta autorski su nastavni primjeri; ne prikazuju implementaciju AWS-a niti platnog sustava u produkciji. Precizna evidencija izvora, pregledanih dijelova i adaptacije starog BA PDF-a nalazi se u sources.md uz predavanje.

- Waldo i sur.: A Note on Distributed Computing, §4.3.
- RFC 5694: P2P Architecture, §2–4.
- Lamport, Shostak, Pease: The Byzantine Generals Problem, uvod.
- AWS: Making retries safe with idempotent APIs.

Nastavni rad (119–120 min): Objasnite što čitati iz kojeg izvora. Literatura nije dodatna lista obveznih proizvoda.

2 Konsenzus i problem bizantskih generala

PLAN — sadržaj još nije razrađen ni pregledan.

2.1 Ishodi učenja

- Razlikovati dogovor, valjanost i završavanje
- Objasniti utjecaj pretpostavki o vremenu i ispadima
- Pokazati zašto jedan identitet nije nužno jedan neovisan glas

2.2 Problem i predznanje

Kako postići dogovor ako neki sudionici lažu, a identiteti se mogu stvarati?

2.3 Pojmovi i pretpostavke

Agreement, validity, termination; Byzantine Generals; FLP intuition; quorum; BFT; Sybil attacks; why classical distributed consensus is not enough for permissionless networks

2.4 Primjer i tijek stanja

Usporediti glasovanje poznatih sudionika s otvorenom mrežom u kojoj jedna osoba stvara više identiteta.

2.5 Ispad i poboljšani dizajn

2.6 Rasprava i provjera razumijevanja

2.7 Sažetak i literatura

3 Temelji blockchaina

PLAN — sadržaj još nije razrađen ni pregledan.

3.1 Ishodi učenja

- Objasniti ulogu sažetka i potpisa
- Pratiti promjenu kroz povezane blokove
- Razlikovati dokaz izmjene od konsenzusa o povijesti

3.2 Problem i predznanje

Kako povezati provjerljive zapise i otkriti naknadnu izmjenu?

3.3 Pojmovi i pretpostavke

Hash functions; hash pointers; Merkle trees; digital signatures; public/private keys; transactions; blocks; append-only ledgers; immutability; blockchain as replicated state

3.4 Primjer i tijek stanja

Promijeniti zapis u nastavnom lancu i pratiti koje se provjere mijenjaju.

3.5 Ispad i poboljšani dizajn

3.6 Rasprava i provjera razumijevanja

3.7 Sažetak i literatura

4 Bitcoin i dokaz rada

PLAN — sadržaj još nije razrađen ni pregledan.

4.1 Ishodi učenja

- Pratiti trošenje UTXO-a
- Objasniti ulogu rada u izboru lanca
- Analizirati račvanje i rizik dvostrukog trošenja

4.2 Problem i predznanje

Kako otvorena mreža bira povijest bez glasovanja po identitetu?

4.3 Pojmovi i pretpostavke

Bitcoin architecture; UTXO model; transactions; mining; PoW; difficulty; longest/heaviest-chain rule; forks; confirmations; double spending; incentives; 51% attacks

4.4 Primjer i tijek stanja

Dvije transakcije pokušavaju potrošiti isti izlaz; pratiti privremene grane i potvrde.

4.5 Ispad i poboljšani dizajn

4.6 Rasprava i provjera razumijevanja

4.7 Sažetak i literatura

5 Ethereum i programabilni blockchain

PLAN — sadržaj još nije razrađen ni pregledan.

5.1 Ishodi učenja

- Usporediti račune i UTXO
- Pratiti promjenu stanja kroz izvršavanje
- Objasniti svrhu gasa i događaja

5.2 Problem i predznanje

Što se mijenja kada transakcija izvršava program?

5.3 Pojmovi i pretpostavke

From Bitcoin to Ethereum; account model vs UTXO; EVM; world state; transactions; gas; smart contracts; blocks; events; Ethereum execution model

5.4 Primjer i tijek stanja

Poziv jednostavnog ugovora mijenja stanje; usporediti uspjeh i prekinuto izvršavanje.

5.5 Ispad i poboljšani dizajn

5.6 Rasprava i provjera razumijevanja

5.7 Sažetak i literatura

6 Dokaz uloga i moderni Ethereum

PLAN — sadržaj još nije razrađen ni pregledan.

6.1 Ishodi učenja

- Razlikovati predlaganje i potvrđivanje
- Objasniti finalnost i kažnjavanje
- Razdvojiti izvršni i konsenzusni sloj

6.2 Problem i predznanje

Tko predlaže blokove i kada odluku smatramo konačnom?

6.3 Pojmovi i pretpostavke

Validators; staking; proposer/attester model; finality; slashing; PoS vs PoW; consensus vs execution; L1 scalability; introduction to L2/rollups

6.4 Primjer i tijek stanja

Pratiti prijedlog bloka, glasove validatora i finalnost; usporediti s PoW modelom.

6.5 Ispad i poboljšani dizajn

6.6 Rasprava i provjera razumijevanja

6.7 Sažetak i literatura

7 Pametni ugovori u Solidityju

PLAN — sadržaj još nije razrađen ni pregledan.

7.1 Ishodi učenja

- Implementirati mali ugovor
- Razlikovati stanje i poziv funkcije
- Postaviti ugovor u razvojno okruženje i pozvati ga

7.2 Problem i predznanje

Kako poslovna radnja postaje provjerljiva promjena stanja ugovora?

7.3 Pojmovi i pretpostavke

Solidity syntax; contract structure; state/storage; mappings; structs; functions; modifiers; events; inheritance/interfaces; deploying and interacting with a simple contract

7.4 Primjer i tijek stanja

Izraditi registar radova/izložaka s vlasnikom, identifikatorom i događajem pri dodavanju.

7.5 Ispad i poboljšani dizajn

7.6 Rasprava i provjera razumijevanja

7.7 Sažetak i literatura

8 Dizajn i sigurnost pametnih ugovora

PLAN — sadržaj još nije razrađen ni pregledan.

8.1 Ishodi učenja

- Prepoznati granice povjerenja
- Napisati test neuspjelog pristupa i promjene stanja
- Obrazložiti sigurnosnu doradu ugovora

8.2 Problem i predznanje

Što se događa kada nepouzdan poziv utječe na zajedničko stanje?

8.3 Pojmovi i pretpostavke

Checks-effects-interactions; reentrancy; access control; integer/economic errors; oracle problem; front-running/MEV; upgradeability; testing; why immutable code changes software engineering

8.4 Primjer i tijek stanja

Testirati dozvole registra; odvojeno pokazati namjerno ranjiv primjer ponovnog ulaska.

8.5 Ispad i poboljšani dizajn

8.6 Rasprava i provjera razumijevanja

8.7 Sažetak i literatura

9 Razvoj Web3 aplikacija

PLAN — sadržaj još nije razrađen ni pregledan.

9.1 Ishodi učenja

- Povezati sučelje i novčanik
- Razlikovati čitanje od slanja transakcije
- Prikazati čekanje, odbijanje i potvrdu

9.2 Problem i predznanje

Kako korisnikov klik postaje potpisana transakcija?

9.3 Pojmovi i pretpostavke

dApp architecture; wallets; signing; RPC; JSON-RPC; ABI; ethers/web3 libraries; reading blockchain state; submitting transactions; events; connecting frontend wallet smart contract

9.4 Primjer i tijek stanja

Frontend registra čita zapise; korisnik potpisuje dodavanje rada i prati ishod.

9.5 Ispad i poboljšani dizajn

9.6 Rasprava i provjera razumijevanja

9.7 Sažetak i literatura

10 Decentralizirana pohrana i IPFS

PLAN — sadržaj još nije razrađen ni pregledan.

10.1 Ishodi učenja

- Objasniti adresiranje sadržajem
- Razlikovati identitet sadržaja i dostupnost
- Povezati metapodatke/datoteku s ugovorom

10.2 Problem i predznanje

Gdje čuvamo datoteke i velike podatke aplikacije?

10.3 Pojmovi i pretpostavke

Why blockchain is poor storage; content addressing; CIDs; DAGs/Merkle DAGs; IPFS nodes; pinning; availability vs permanence; IPFS + Ethereum; metadata and NFTs as a practical example

10.4 Primjer i tijek stanja

Dodati sliku i metapodatke rada na IPFS, a referencu u registar; provjeriti dostupnost.

10.5 Ispad i poboljšani dizajn

10.6 Rasprava i provjera razumijevanja

10.7 Sažetak i literatura

11 Arhitekture blockchain aplikacija

PLAN — sadržaj još nije razrađen ni pregledan.

11.1 Ishodi učenja

- Raspodijeliti odgovornosti on-chain/off-chain
- Identificirati preostalo povjerenje u usluge
- Obrazložiti cjelovitu arhitekturu

11.2 Problem i predznanje

Koji dio aplikacije treba zajednički konsenzus, a koji ne?

11.3 Pojmovi i pretpostavke

On-chain vs off-chain computation/storage; oracles; indexers; IPFS; backend services; identity; tokens; NFTs; DAOs; supply chain; provenance; DeFi; hybrid architectures

11.4 Primjer i tijek stanja

Dodati pretragu/indexer i prikaz izvora podataka; usporediti scenarije provenijencije, DAO-a i DeFi-ja.

11.5 Ispad i poboljšani dizajn

11.6 Rasprava i provjera razumijevanja

11.7 Sažetak i literatura

12 Skaliranje, ograničenja i kada koristiti blockchain

PLAN — sadržaj još nije razrađen ni pregledan.

12.1 Ishodi učenja

- Usporediti L1, L2 i centraliziranu izvedbu
- Procijeniti pretpostavke mostova i privatnosti
- Obraniti izbor ili odbacivanje blockchaina

12.2 Problem i predznanje

Treba li ovoj aplikaciji blockchain i koju cijenu zbog toga plaćamo?

12.3 Pojmovi i pretpostavke

L2s and rollups; bridges; interoperability; privacy; performance/cost trade-offs; decentralization–scalability–security trade-offs; permissioned blockchains; evaluating whether an application actually needs blockchain

12.4 Primjer i tijek stanja

Usporediti isti registar na L1, L2 i običnoj bazi; obrazložiti trošak, povjerenje i dostupnost.

12.5 Ispad i poboljšani dizajn

12.6 Rasprava i provjera razumijevanja

12.7 Sažetak i literatura